HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN: THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

ĐỀ TÀI: XÂY DỰNG WEBSITE ĐỌC TRUYỆN TRỰC TUYẾN METRUYEN

Sinh viên thực hiện: Nguyễn Thành Nam

Mã sinh viên: B23DCCN587

Nhóm: 11

Giảng viên hướng dẫn: Kim Ngọc Bách

Hà Nội - 2026

Mục Lục

1. Nội dung công việc đã thực hiện trong tuần 3

1.1. Hoàn thiện API Category CRUD 3

1.2. Hoàn thiện API Comic CRUD + upload ảnh 3

1.3. Hoàn thiện API Chapter upload theo Comic 4

2. CHI TIẾT TRIỂN KHAI 4

3. KẾT QUẢ KIỂM THỬ 6

3.1. Test Case 1: Tạo Category mới (Admin) 6

3.2. Test Case 2: Tạo Comic mới có upload coverImage 7

3.3. Test Case 3: Lấy danh sách comics 8

3.4. Test Case 4: Upload chương mới cho comic (đa ảnh) 9

4. KẾT QUẢ ĐẠT ĐƯỢC 10

5. KẾ HOẠCH TUẦN 5 10

TÀI LIỆU THAM KHẢO 11

1. Nội dung công việc đã thực hiện trong tuần

Trong tuần trước, em đã Hoàn thiện API (CRUD) quản lý thể loại (Category) và quản lý Truyện (Comic), Hoàn thiện API upload Chương (Chapter) theo comic. Đồng thời Nghiên cứu thư viện Multer và tích hợp Cloudinary để xử lý upload hình ảnh

1.1. Hoàn thiện API Category CRUD

- Công nghệ/thư viện sử dụng:

- Express Router: Tổ chức endpoint theo module [1].
- Mongoose: Thao tác dữ liệu MongoDB [2].
- JWT Middleware: Bảo vệ route quản trị .

- Kết quả chính:

- Public endpoints:

- `GET /api/categories` - Lấy danh sách thể loại
- `GET /api/categories/:slug` - Lấy chi tiết thể loại

- Admin endpoints (yêu cầu token + role admin):

- `POST /api/categories` - Tạo thể loại mới
- `PUT /api/categories/:id` - Cập nhật thể loại
- `DELETE /api/categories/:id` - Xóa thể loại

1.2. Hoàn thiện API Comic CRUD + upload ảnh

- Công nghệ/thư viện sử dụng:

- Multer: Nhận file upload (memoryStorage) [3].
- Cloudinary: Lưu trữ ảnh cover/banner trên cloud [4].
- Middleware verifyToken + isAdmin: Phân quyền thao tác tạo/sửa/xóa

- Kết quả chính:

- Public endpoints:

- `GET /api/comics` - Danh sách truyện (search/filter/pagination)
- `GET /api/comics/slug/:slug` - Chi tiết truyện theo slug
- `GET /api/comics/:id` - Chi tiết truyện theo ID
- `POST /api/comics/:id/view` - Tăng lượt xem

- Admin endpoints:

- `POST /api/comics` - Tạo truyện (kèm upload `coverImage`, `bannerImage`)
- `PUT /api/comics/:id` - Cập nhật truyện (kèm upload ảnh)
- `DELETE /api/comics/:id` - Xóa truyện

1.3. Hoàn thiện API Chapter upload theo Comic

- Công nghệ/thư viện sử dụng:

- Multer uploadMultiple: Upload nhiều ảnh trang truyện theo field `pages` [3].
- Middleware verifyToken + isAdmin: Chỉ quản trị viên được upload chương.
- Chapter Controller: Tạo chương và liên kết với comic.

- Kết quả chính:

- Admin endpoint:

- `POST /api/comics/:comicId/chapters` - Tạo chương mới và upload tối đa 100 ảnh trang

2. CHI TIẾT TRIỂN KHAI

- `backend/src/routes/categoryRoutes.js`: Khai báo toàn bộ endpoint `GET/POST/PUT/DELETE` cho Category; tách rõ route public và route admin, đồng thời gắn middleware `verifyToken` + `isAdmin` để chặn truy cập trái phép vào chức năng quản trị.

- `backend/src/controllers/categoryController.js`: Chứa logic nghiệp vụ Category CRUD, bao gồm kiểm tra dữ liệu đầu vào, truy vấn MongoDB bằng Mongoose, xử lý lỗi không tìm thấy bản ghi và trả response chuẩn cho frontend.

- `backend/src/routes/comicRoutes.js`: Định tuyến nhóm API Comic, gồm endpoint public (danh sách, chi tiết, tăng view) và endpoint admin (tạo/sửa/xóa); tích hợp middleware upload ảnh `coverImage`, `bannerImage` và route upload chapter theo `comicId`.

- `backend/src/controllers/comicController.js`: Xử lý đầy đủ nghiệp vụ Comic CRUD: lọc/tìm kiếm/phân trang danh sách truyện, lấy chi tiết truyện, tạo/cập nhật truyện kèm upload ảnh lên Cloudinary, đồng thời cập nhật thống kê lượt xem.

- `backend/src/controllers/chapterController.js`: Thực hiện nghiệp vụ tạo chapter cho từng comic; nhận nhiều ảnh trang truyện (`pages`), upload theo batch, gắn thứ tự trang, lưu chapter mới vào DB và liên kết chapter với comic tương ứng.

- `backend/src/middlewares/upload.js`: Cấu hình Multer theo `memoryStorage` để tối ưu luồng upload cloud; kiểm tra MIME type hợp lệ, giới hạn kích thước file (5MB/file), và cung cấp các hàm `uploadSingle`, `uploadMultiple`, `uploadFields` cho từng kiểu API.

- `backend/src/config/cloudinary.js`: Khởi tạo kết nối Cloudinary từ biến môi trường, định nghĩa helper upload ảnh (trả về `url`, `publicId`, kích thước ảnh) và helper xóa ảnh, giúp controller tái sử dụng mà không lặp code.

- `backend/src/models/Category.js`: Định nghĩa schema Category (name, slug, description, order, isActive...), ràng buộc dữ liệu và index cần thiết để phục vụ phân loại truyện và truy vấn danh mục ổn định.

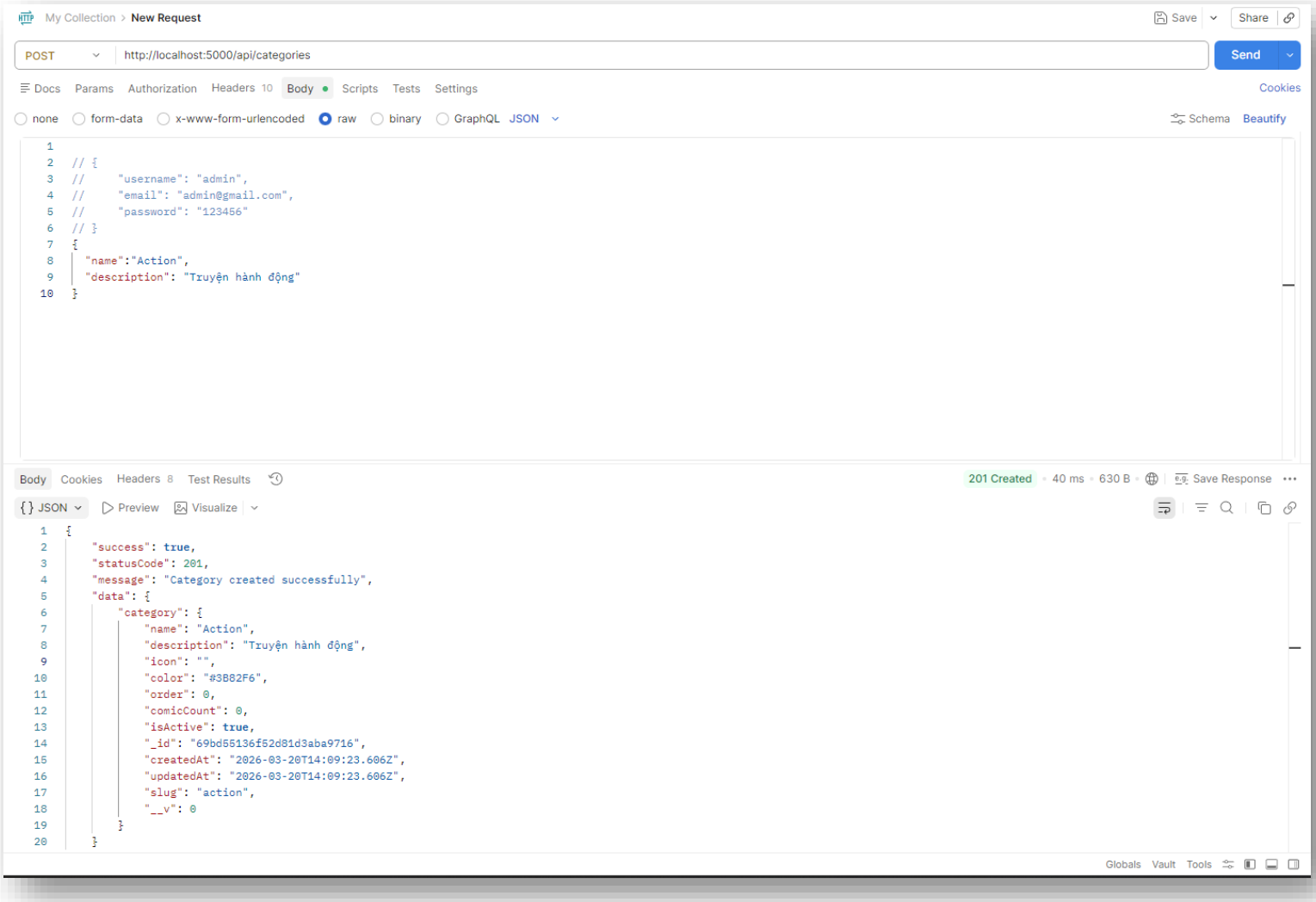
- `backend/src/models/Comic.js`: Định nghĩa schema Comic gồm thông tin cơ bản, ảnh bìa/banner, category, trạng thái phát hành và thống kê; hỗ trợ lưu metadata phục vụ hiển thị danh sách/trang chi tiết truyện.

- `backend/src/models/Chapter.js`: Định nghĩa schema Chapter theo quan hệ với Comic (`comicId`), lưu thông tin số chương, tiêu đề và mảng `pages` (URL/publicId/thứ tự), bảo đảm dữ liệu đủ để render trình đọc truyện.

- `backend/.env`: Lưu cấu hình nhạy cảm cần bảo mật (MongoDB URI, JWT Secret, Cloudinary keys) theo môi trường chạy, giúp bảo mật tốt hơn và dễ chuyển đổi giữa local/dev/production.

3. KẾT QUẢ KIỂM THỬ

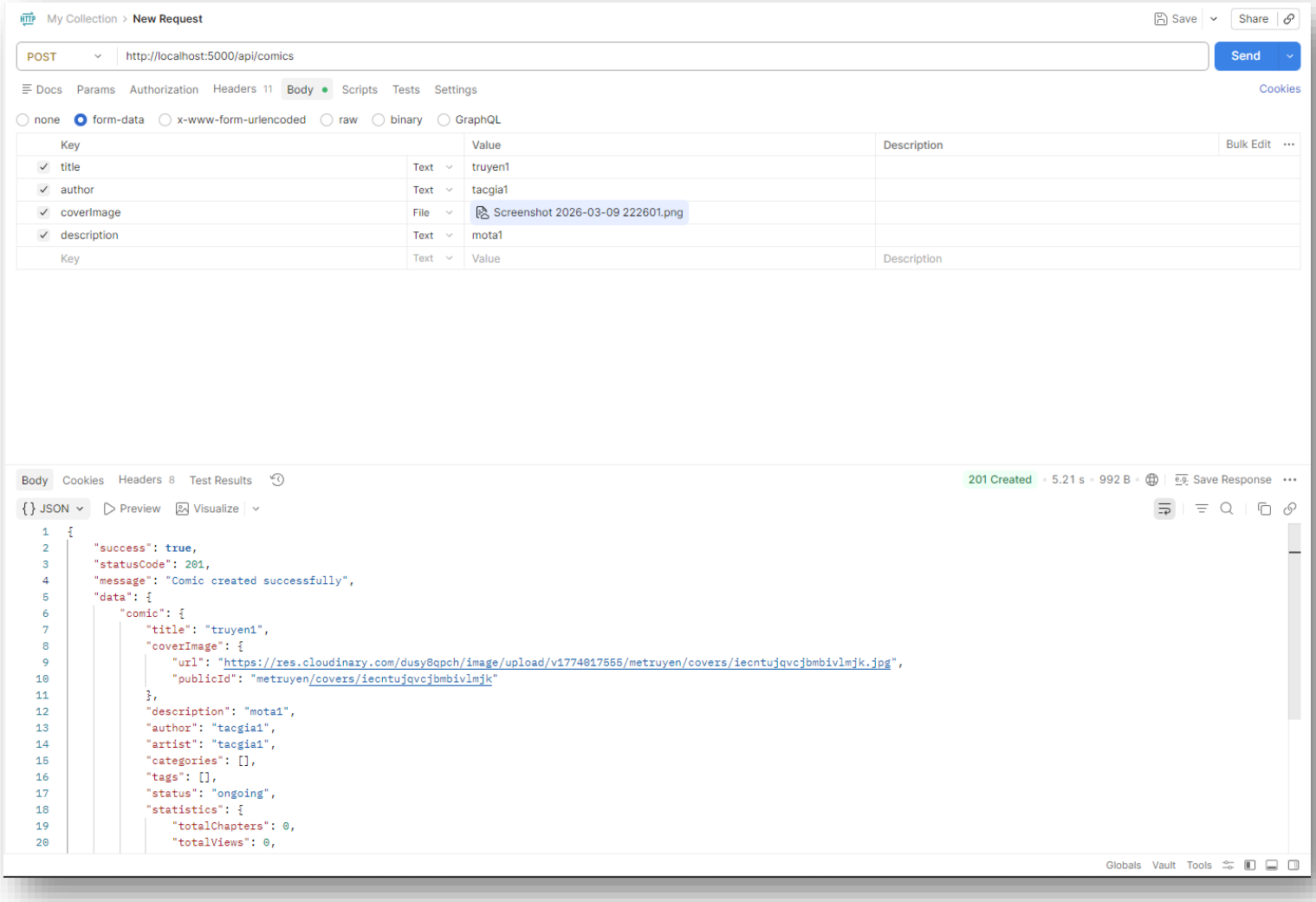
3.1. Test Case 1: Tạo Category mới (Admin)



- Kết quả kiểm thử thực tế:

- API trả về `201 Created`.
- Category mới được tạo thành công với `name = "Action"` và sinh `slug` tương ứng.
- Dữ liệu category được ghi nhận trong collection `categories`.

3.2. Test Case 2: Tạo Comic mới có upload coverImage



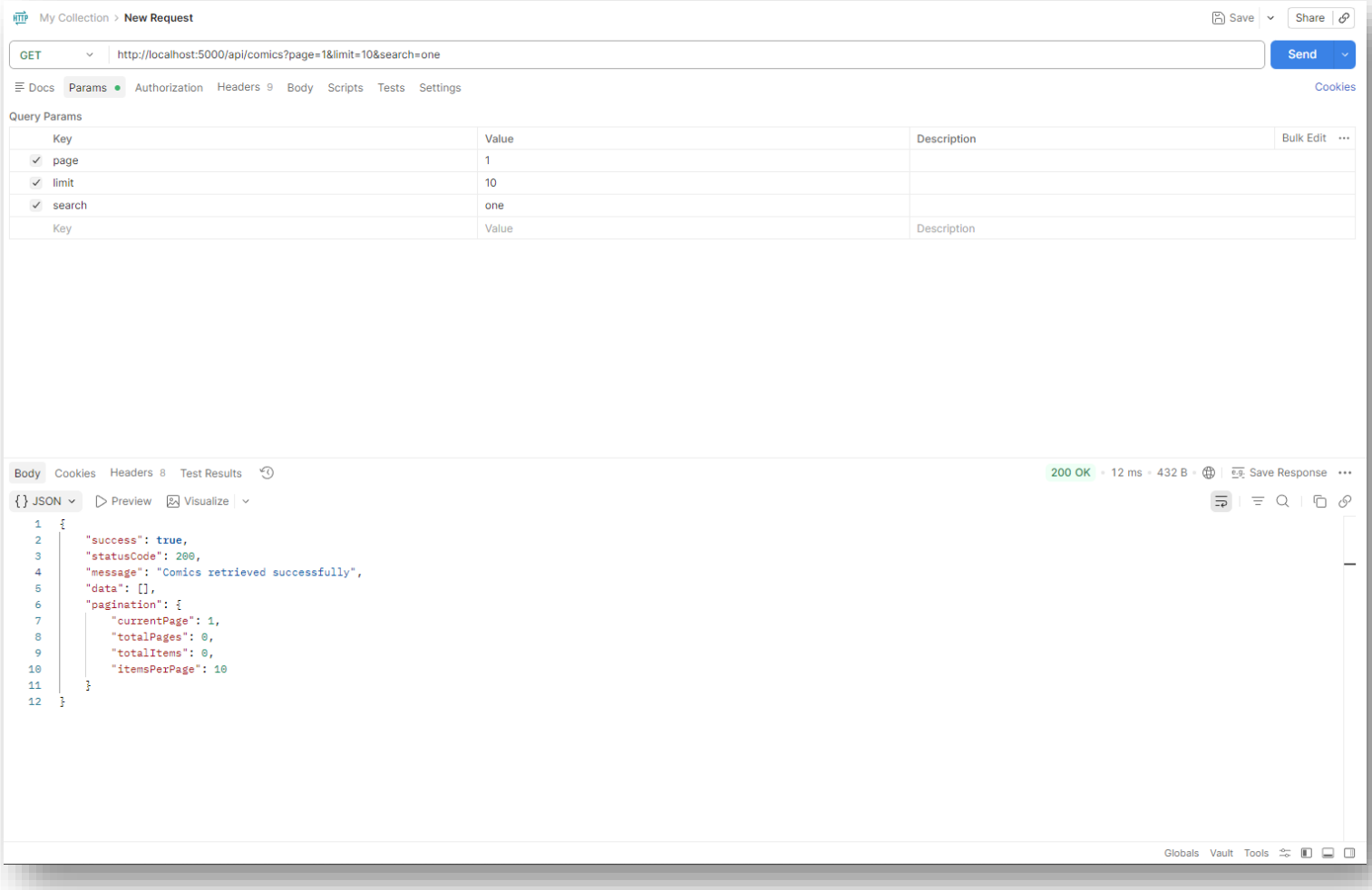
- Kết quả kiểm thử thực tế:

-API trả về `201 Created`.

- Ảnh `coverImage` upload thành công lên Cloudinary.

- Response chứa URL ảnh cloud hợp lệ và comic mới được lưu trong collection `comics`.

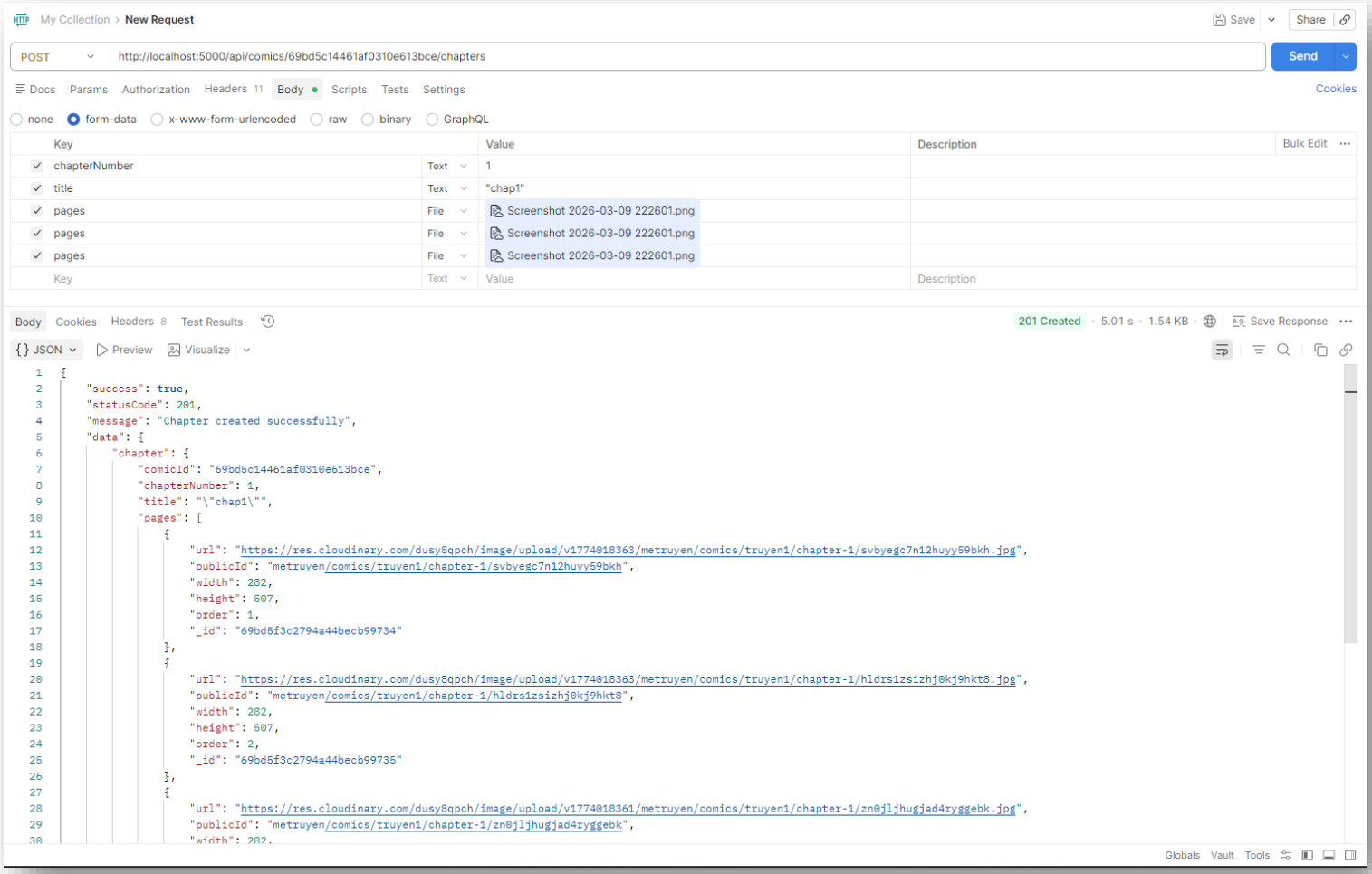
3.3. Test Case 3: Lấy danh sách comics



- Kết quả kiểm thử thực tế:

- API trả về `200 OK`.
- Danh sách truyện trả về đúng theo từ khóa tìm kiếm (`search=one`).
- Trường phân trang (`currentPage`, `totalPages`, `totalItems`, `itemsPerPage`) có dữ liệu hợp lệ.

3.4. Test Case 4: Upload chương mới cho comic (đa ảnh)



- Kết quả kiểm thử thực tế:

- API trả về `201 Created`.
- Nhiều ảnh trang (`pages`) được upload thành công, giữ đúng thứ tự trang.
- Chapter mới được tạo và liên kết đúng với `comicId` tương ứng.

4. KẾT QUẢ ĐẠT ĐƯỢC

- Hoàn thiện Category CRUD API
- Hoàn thiện Comic CRUD API
- Hoàn thiện Chapter upload API theo comic
- Tích hợp Multer cho upload file ảnh
- Tích hợp Cloudinary để lưu ảnh cover

5. KẾ HOẠCH TUẦN SAU

- Khởi tạo môi trường Frontend (ReactJS/Vite/TailwindCSS).
- Xây dựng Layout chung (Header, Footer, Navigation).
- Phát triển giao diện Đăng nhập & Đăng ký.
- Cấu hình bảo mật: PrivateRoute (cho member), AdminRoute (cho admin), JWT token storage và management bằng Zustand.

TÀI LIỆU THAM KHẢO

[1] "Express routing." Accessed: Mar. 20, 2026. [Online]. Available: https://expressjs.com/en/guide/routing.html

[2] "Mongoose v9.3.1: Getting Started." Accessed: Mar. 20, 2026. [Online]. Available: https://mongoosejs.com/docs/

[3] *expressjs/multer*. (Mar. 20, 2026). JavaScript. expressjs. Accessed: Mar. 20, 2026. [Online]. Available: https://github.com/expressjs/multer

[4] "Node.js SDK – Node.js Upload + Image, Video Transformations | Documentation." Accessed: Mar. 20, 2026. [Online]. Available: https://cloudinary.com/documentation/node_integration